EFREI Paris – Cycle ingénieur
Machine Learning I – Année 2026–2027

Classification de la qualité du vin

à partir de ses propriétés physico-chimiques

Rapport technique

Auteurs : *[Nom Prénom]*
[Nom Prénom]
[Nom Prénom]

Encadrants : Ahmed Ghazi Blaiech, Ahmad Tay, Hassan Shraim

Code : notebook `projet_vin.ipynb` (Python, scikit-learn)

Résumé

Ce rapport présente une chaîne complète d'apprentissage supervisé visant à prédire si un vin est « bon » (note d'experts ≥ 7) à partir de onze mesures physico-chimiques. Les données proviennent du jeu *Wine Quality* de l'UCI (vins rouges et blancs portugais *Vinho Verde*). Après nettoyage (suppression de 1 177 doublons, détection des valeurs aberrantes par la règle de Tukey, encodage), nous obtenons 5 320 vins, dont seulement 19 % de « bons ». Ce fort déséquilibre guide l'ensemble des choix méthodologiques : découpage stratifié, pondération des classes et utilisation du score F_1 et de l'AUC plutôt que de l'exactitude. Une sélection de variables par élimination récursive (RFE) confirme le rôle dominant du degré d'alcool et de l'acidité volatile. Cinq classifieurs sont comparés (régression logistique, k plus proches voisins, arbre de décision, Naive Bayes gaussien, SVM à noyau RBF), d'abord par validation croisée à 5 plis sur l'ensemble d'apprentissage, avec un diagnostic biais/variance, puis après optimisation de leurs hyperparamètres par recherche en grille et recherche aléatoire. Le jeu de test n'est utilisé qu'une seule fois, pour l'évaluation finale. Le SVM à noyau RBF obtient les meilleures performances ($F_1 = 0{,}54$, AUC $= 0{,}85$, rappel $= 0{,}82$ sur le test) et le meilleur compromis biais/variance. Les performances, modestes en valeur absolue, sont discutées au regard de la subjectivité de la note et du chevauchement des classes.

Table des matières

1 Introduction **2**
1.1 Contexte . . . 2
1.2 Problématique et objectifs . . . 2
1.3 Organisation du rapport . . . 2

2 Données **2**

3 Prétraitement **3**
3.1 Valeurs manquantes et doublons . . . 3
3.2 Valeurs aberrantes . . . 3
3.3 Encodage . . . 4

4 Variable cible et déséquilibre des classes **4**

5 Analyse exploratoire **5**

6 Protocole expérimental **6**
6.1 Découpage et standardisation . . . 6
6.2 Validation croisée et usage unique du test . . . 7

7 Sélection de variables **7**
7.1 Méthode : élimination récursive (RFE) . . . 7
7.2 Résultats . . . 7
7.3 Nombre de variables retenues . . . 8

8 Modèles **8**

9 Évaluation des modèles par validation croisée **10**
9.1 Métriques . . . 10
9.2 Résultats des modèles par défaut . . . 10
9.3 Diagnostic biais / variance . . . 11

10 Optimisation des hyperparamètres **12**
10.1 Méthodologie 12
10.2 Résultats 13

11 Évaluation finale sur le jeu de test **13**

12 Discussion **15**

13 Conclusion **16**

A Reproductibilité **16**

1 Introduction

1.1 Contexte

La qualité d'un vin est traditionnellement évaluée par des dégustateurs experts, ce qui est coûteux, lent et en partie subjectif. Or de nombreuses grandeurs physico-chimiques (acidité, sucre résiduel, teneur en alcool, sulfites, etc.) sont mesurées en routine lors de la certification. La question est donc de savoir si ces mesures objectives suffisent à prédire le jugement des experts. Aucune formule analytique ne relie ces mesures à la qualité perçue : c'est un cas typique où l'apprentissage automatique permet d'extraire une règle de décision à partir d'exemples étiquetés.

1.2 Problématique et objectifs

Conformément au sujet, le problème initialement ordinal (note de 0 à 10) est reformulé en **classification binaire** : un vin est « bon » si sa note est supérieure ou égale à 7, « pas bon » sinon. Les objectifs sont :

1. préparer et analyser les données (qualité, distributions, corrélations, équilibre des classes) ;
2. sélectionner les variables pertinentes avec une méthode vue en cours ;
3. comparer cinq modèles de natures différentes (linéaire / non linéaire, paramétrique / non paramétrique, probabiliste / géométrique) ;
4. évaluer ces modèles avec des métriques adaptées au déséquilibre et identifier le meilleur compromis biais/variance ;
5. optimiser leurs hyperparamètres et mesurer la performance finale sur des données jamais vues.

1.3 Organisation du rapport

La section 2 décrit les données et la section 3 leur prétraitement. La section 4 définit la cible et traite le déséquilibre des classes, puis la section 5 présente l'analyse exploratoire. Le protocole expérimental est détaillé en section 6 et la sélection de variables en section 7. Les sections 8 à 11 portent sur les modèles, leur évaluation, leur optimisation et l'évaluation finale. La section 12 discute les limites et perspectives.

2 Données

Nous utilisons le jeu de données *Wine Quality* de l'UCI Machine Learning Repository [1], composé de deux fichiers : 1 599 vins rouges et 4 898 vins blancs, soit 6 497 observations. Chaque vin est décrit par onze variables continues issues de tests physico-chimiques et par une note médiane attribuée par au moins trois experts (tableau 1). Les notes observées vont de 3 à 9.

Table 1 – Variables du jeu de données.

Variable	Description	Unité
`fixed acidity`	acidité fixe (acide tartrique)	g/L
`volatile acidity`	acidité volatile (acide acétique)	g/L
`citric acid`	acide citrique	g/L
`residual sugar`	sucre résiduel	g/L
`chlorides`	chlorures (sel)	g/L
`free sulfur dioxide`	SO_2 libre	mg/L
`total sulfur dioxide`	SO_2 total	mg/L
`density`	densité	g/cm^3
`pH`	pH	–
`sulphates`	sulfates	g/L
`alcohol`	teneur en alcool	% vol.
`type`	couleur (ajoutée lors de la fusion)	rouge / blanc
`quality`	note des experts	0 à 10

Les deux fichiers ont été fusionnés afin de disposer de davantage d'exemples, en particulier pour les vins rouges, moins nombreux. Une variable `type` conserve l'information de couleur, ce qui permet au modèle de l'exploiter si elle est utile. Les statistiques descriptives révèlent deux points importants pour la suite : (i) des échelles très hétérogènes (densité proche de 1, SO_2 total jusqu'à 440 mg/L), qui imposent une standardisation pour les méthodes fondées sur des distances ; (ii) des maxima très éloignés du troisième quartile pour plusieurs variables (`residual sugar`, `chlorides`, `free sulfur dioxide`), signe de valeurs extrêmes.

3 Prétraitement

3.1 Valeurs manquantes et doublons

Aucune valeur manquante n'est présente : aucune imputation n'est nécessaire. En revanche, le jeu contient **1 177 doublons exacts** (18 % des lignes), soit exactement la somme des doublons internes à chaque fichier (240 rouges et 937 blancs) : la fusion n'en crée aucun. Ces doublons sont supprimés *avant* le découpage apprentissage/test. Sans cette précaution, deux copies d'un même vin pourraient se retrouver de part et d'autre du découpage, et le modèle serait évalué sur des exemples déjà vus, ce qui gonflerait artificiellement les scores. Il reste **5 320 vins**.

3.2 Valeurs aberrantes

Les valeurs aberrantes sont détectées par la règle de Tukey [3], qui définit les moustaches d'une boîte à moustaches. Avec Q_1 et Q_3 les premier et troisième quartiles et $\mathrm{IQR} = Q_3 - Q_1$ l'écart interquartile, une valeur x est signalée si

$$x < Q_1 - 1{,}5\,\mathrm{IQR} \quad \text{ou} \quad x > Q_3 + 1{,}5\,\mathrm{IQR}. \tag{1}$$

Fondée sur les quartiles, cette règle est robuste : les valeurs extrêmes ne déplacent pas les bornes, contrairement à un critère fondé sur la moyenne et l'écart-type. Elle ne suppose pas non plus de distribution gaussienne, hypothèse non vérifiée ici.

Les vins rouges et blancs ayant des compositions chimiques différentes, l'IQR a été recalculé *séparément pour chaque couleur* (figure 1). Le nombre total de valeurs signalées varie peu (environ 1 370 sur le jeu mélangé contre 1 340 par couleur), mais sa répartition change nettement. Pour `fixed acidity` et `volatile acidity`, le nombre de valeurs signalées est divisé par deux environ ($304 \to 147$ et $279 \to 152$) : une partie de ces « anomalies » ne traduisait que l'écart entre rouges et blancs. À l'inverse, le mélange masquait des valeurs atypiques pour `density` ($3 \to 41$) ou `total sulfur dioxide` ($10 \to 59$).

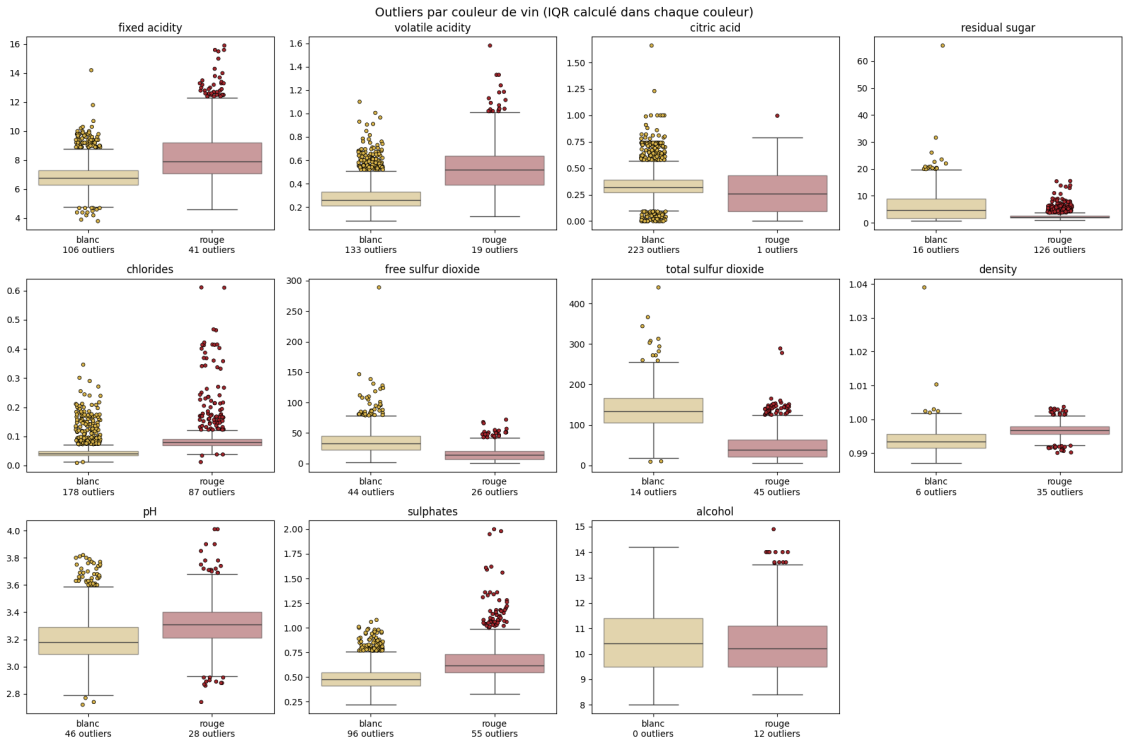


Figure 1 – Valeurs aberrantes détectées par la règle de Tukey, calculée séparément pour les vins blancs et rouges.

Dans chaque couleur, les valeurs signalées représentent le plus souvent 1 à 6 % des vins et forment des queues continues de distribution, ce qui correspond à des vins atypiques plutôt qu'à des erreurs de mesure. Le taux élevé observé pour le sucre résiduel des rouges (9,3 %) s'explique par une distribution très resserrée autour de 2 g/L, qui rend la règle très stricte ; les valeurs signalées (4 à 15 g/L) restent plausibles. Nous avons donc **conservé toutes les observations**, leur influence étant limitée par la standardisation (section 6).

3.3 Encodage

La variable `type` est la seule variable catégorielle. Ayant deux modalités sans ordre, elle est codée en binaire (blanc = 0, rouge = 1). Un encodage *one-hot* produirait deux colonnes parfaitement redondantes.

4 Variable cible et déséquilibre des classes

La cible est définie par $y = \mathbb{1}[\texttt{quality} \geq 7]$. Elle est fortement déséquilibrée : **81 % de vins « pas bons » et 19 % de « bons »** (figure 2).

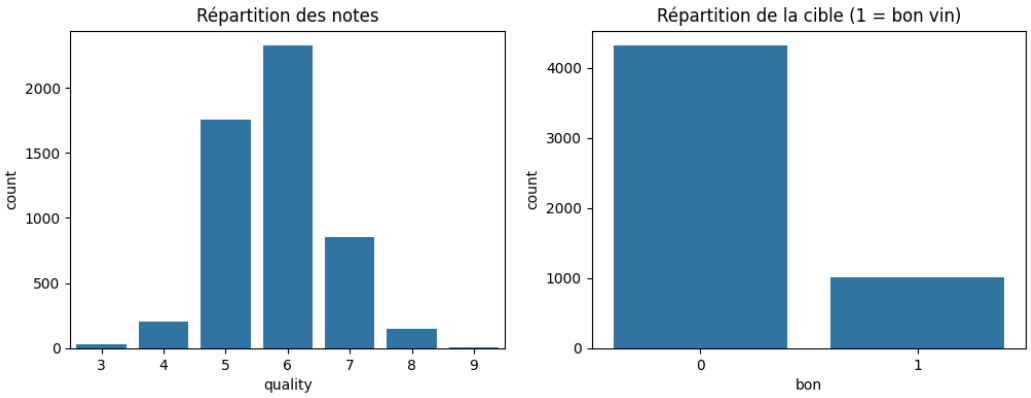


Figure 2 – Répartition des notes (gauche) et de la cible binaire (droite).

Ce déséquilibre rend l'exactitude (*accuracy*) trompeuse : un classifieur trivial prédisant toujours « pas bon » atteindrait 81 % d'exactitude sans détecter aucun bon vin. Trois mesures ont été prises.

— **Découpage stratifié** : les ensembles d'apprentissage et de test conservent la proportion de 19 % de bons vins.

— **Pondération des classes** (`class_weight="balanced"`), lorsque le modèle le permet. Le poids de la classe c vaut

$$w_c = \frac{n}{K\, n_c}, \tag{2}$$

où n est le nombre d'exemples, $K = 2$ le nombre de classes et n_c l'effectif de la classe. Sur l'ensemble d'apprentissage ($n = 4\,256$, dont 807 bons vins), on obtient $w_{\text{bon}} \approx 2{,}64$ et $w_{\text{pas bon}} \approx 0{,}62$: une erreur sur un bon vin coûte environ quatre fois plus cher. C'est le même principe que la pondération des observations dans AdaBoost.

— **Métriques adaptées** : le score F_1 de la classe positive est la métrique principale, complétée par l'AUC, indépendante du seuil de décision (section 9).

Une alternative, discutée en section 12, consiste à conserver un apprentissage non pondéré et à ajuster le seuil de décision à partir de la courbe ROC.

5 Analyse exploratoire

Distributions. Les histogrammes montrent que plusieurs variables (`chlorides`, `residual sugar`, `sulphates`) sont fortement asymétriques à droite. L'hypothèse gaussienne du classifieur Naive Bayes est donc violée, ce qui laisse prévoir des performances moindres pour ce modèle.

Corrélations. Le coefficient de corrélation de Pearson entre chaque variable et la cible place nettement en tête l'alcool ($r = +0{,}42$), suivi de la densité ($-0{,}29$), des chlorures ($-0{,}16$) et de l'acidité volatile ($-0{,}14$) ; toutes les autres variables ont $|r| < 0{,}09$. Plusieurs variables explicatives sont fortement corrélées entre elles : SO_2 libre et total (0,72), densité et alcool ($-0{,}67$), densité et sucre résiduel (0,52). Ces dépendances sont physiquement attendues (la densité d'un vin dépend de sa teneur en alcool et en sucre) et violent l'hypothèse d'indépendance de Naive Bayes.

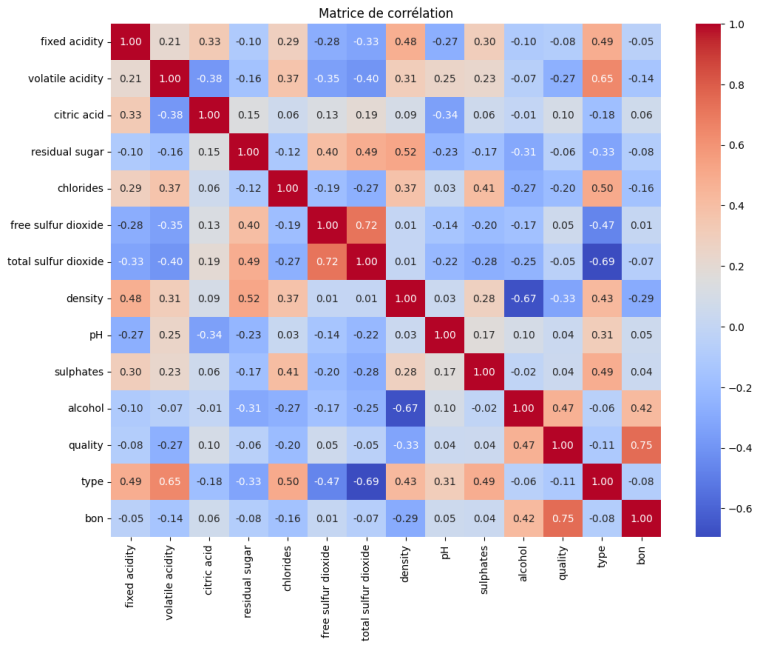


Figure 3 – Matrice des corrélations de Pearson (variables explicatives, note et cible).

Boîtes à moustaches par classe. Pour les quatre variables les plus corrélées à la cible (figure 4), les bons vins présentent une teneur en alcool nettement plus élevée, ainsi qu'une densité, une acidité volatile et une teneur en chlorures plus faibles. L'alcool est la variable qui sépare le mieux les deux classes. Ces observations sont cohérentes avec l'œnologie : l'acidité volatile, principalement due à l'acide acétique, est un défaut gustatif classique.

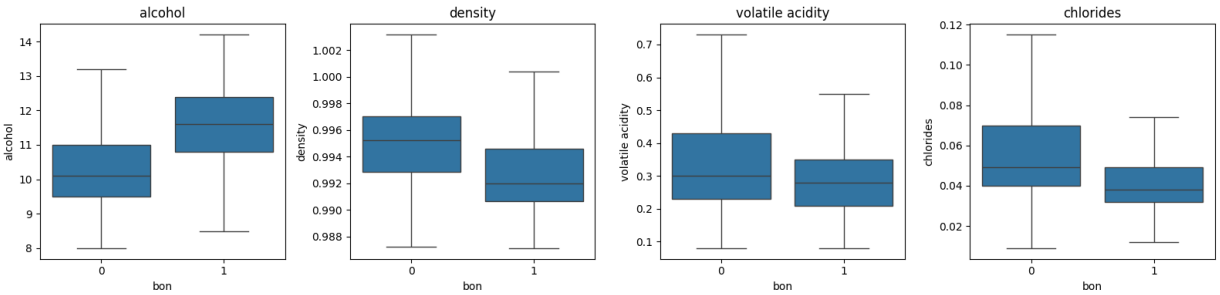


Figure 4 – Distribution des quatre variables les plus corrélées à la cible, selon la classe (0 = pas bon, 1 = bon).

6 Protocole expérimental

6.1 Découpage et standardisation

Les données sont découpées en un ensemble d'apprentissage (80 %, 4 256 vins dont 807 bons) et un ensemble de test (20 %, 1 064 vins dont 202 bons), de façon stratifiée et reproductible (`random_state=42`). Les variables sont ensuite standardisées (*z-score*) :

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \tag{3}$$

où μ_j et σ_j sont la moyenne et l'écart-type de la variable j **calculés sur l'ensemble d'apprentissage uniquement**, puis appliqués tels quels au test. La standardisation est préférée à la normalisation min-max car les variables ont des unités très différentes, et parce que la normalisation

min-max dépend des valeurs extrêmes (un sucre résiduel de 66 g/L comprimerait tous les autres vins).

6.2 Validation croisée et usage unique du test

Toute décision (sélection de variables, choix des hyperparamètres, choix du modèle) est prise par **validation croisée stratifiée à 5 plis sur l'ensemble d'apprentissage**. Pour les tableaux de résultats, matrices de confusion et courbes ROC des modèles par défaut, nous utilisons des prédictions *hors-pli* (*out-of-fold*) : chaque vin de l'apprentissage est prédit par un modèle entraîné sur les quatre autres plis, qui ne l'a donc jamais vu. Le **jeu de test n'est utilisé qu'une seule fois**, pour l'évaluation finale des modèles optimisés. Utiliser le test pour choisir un modèle reviendrait à l'optimiser pour cet échantillon particulier, et le score obtenu ne serait plus une estimation non biaisée de l'erreur de généralisation. La figure 5 résume ce protocole.

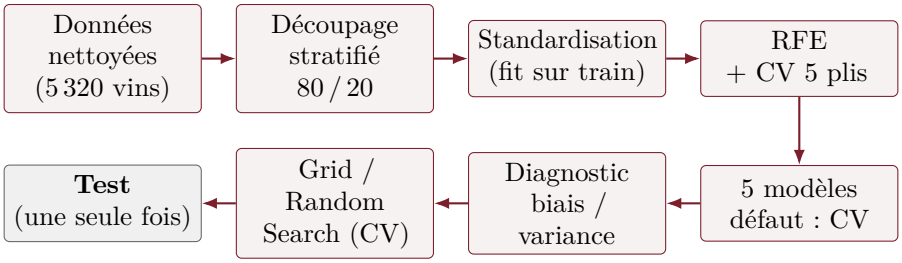


Figure 5 – Protocole expérimental. Toutes les étapes en rouge n'utilisent que l'ensemble d'apprentissage.

7 Sélection de variables

7.1 Méthode : élimination récursive (RFE)

Parmi les trois familles de méthodes de sélection (filtre, *wrapper*, embarquée), nous retenons une méthode *wrapper*, l'élimination récursive de variables (RFE, algorithme 1). Contrairement à un filtre, qui évalue chaque variable isolément, la RFE évalue les variables *conjointement* au sein d'un modèle. Elle tient donc compte des redondances et des effets de combinaison. L'estimateur sous-jacent est une régression logistique pondérée (`class_weight="balanced"`) entraînée sur les données standardisées ; l'importance d'une variable est mesurée par la valeur absolue de son coefficient.

Algorithme 1 : Élimination récursive de variables (RFE)

Input : ensemble de variables $S = \{1, \dots, p\}$, données d'apprentissage
Output : rang de chaque variable

$r \leftarrow p$;
while $|S| > 1$ **do**
  entraîner une régression logistique sur les variables de S;
  $j^\star \leftarrow \arg\min_{j \in S} |\beta_j|$; `// variable la moins utile`
  $\text{rang}(j^\star) \leftarrow r$; $S \leftarrow S \setminus \{j^\star\}$; $r \leftarrow r - 1$;
$\text{rang}(\text{dernière variable restante}) \leftarrow 1$;

7.2 Résultats

Le classement obtenu figure au tableau 2. L'alcool est classé premier, en accord avec l'analyse de corrélation, suivi de l'acidité volatile. Plusieurs différences avec le classement par corrélation sont instructives :

— la **densité**, deuxième variable la plus corrélée à la cible, n'est que cinquième : sa forte dépendance à l'alcool et au sucre la rend largement redondante une fois l'alcool présent dans le modèle ;

— les **chlorures**, troisièmes en corrélation, sont dixièmes pour la même raison ;

— l'**acidité fixe** et le **pH**, peu corrélés à la cible, sont classés troisième et quatrième : ils deviennent informatifs en combinaison avec d'autres variables, ce qu'un filtre univarié ne peut pas détecter.

Une limite de ce classement est qu'il repose sur un modèle linéaire : une variable dont l'effet est uniquement non linéaire peut être sous-évaluée.

Table 2 – Classement RFE et corrélation de Pearson avec la cible.

Rang	Variable	r	Rang	Variable	r
1	alcohol	+0,42	7	sulphates	+0,04
2	volatile acidity	−0,14	8	total sulfur dioxide	−0,07
3	fixed acidity	−0,05	9	free sulfur dioxide	+0,01
4	pH	+0,05	10	chlorides	−0,16
5	density	−0,29	11	type	−0,08
6	residual sugar	−0,08	12	citric acid	+0,06

7.3 Nombre de variables retenues

Pour choisir le nombre de variables, nous évaluons par validation croisée le F_1 d'une régression logistique utilisant les k premières variables du classement, pour $k = 1, \ldots, 12$ (figure 6). L'alcool seul atteint déjà $F_1 = 0{,}510$. Le score progresse jusqu'à environ 0,53 avec deux variables, puis atteint un plateau entre 0,53 et 0,54 à partir de sept variables (maximum 0,540 pour $k = 11$, 0,539 pour $k = 12$). Les écarts sont de l'ordre du centième, donc inférieurs à la variabilité entre plis. Avec seulement 12 variables pour plus de 4 000 exemples, le risque lié à la dimension est négligeable : nous conservons **les 12 variables**.

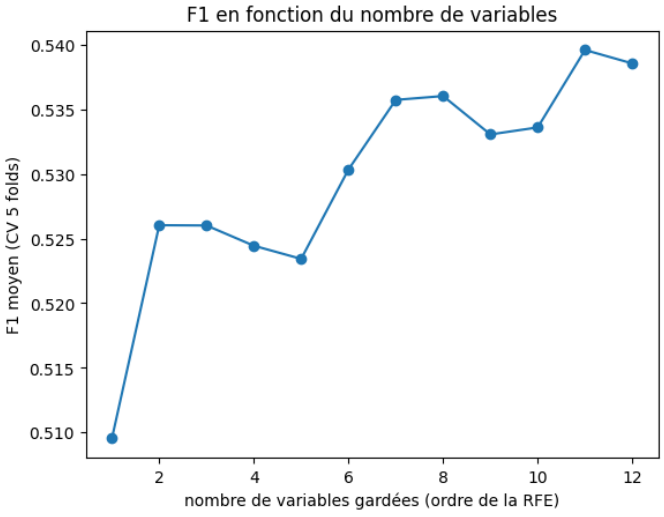


Figure 6 – Score F_1 moyen (validation croisée, régression logistique) selon le nombre de variables retenues dans l'ordre de la RFE.

8 Modèles

Les cinq modèles demandés couvrent des hypothèses très différentes, ce qui rend leur comparaison informative. On note $\mathbf{x} \in \mathbb{R}^p$ le vecteur des variables standardisées et $y \in \{0, 1\}$ la

classe.

Régression logistique. Modèle linéaire probabiliste :

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}. \tag{4}$$

Les paramètres minimisent l'entropie croisée binaire, éventuellement pondérée par w_c, augmentée d'une pénalité ℓ_2 (Ridge) dont la force est inversement proportionnelle à l'hyperparamètre C :

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|_2^2 \ - \ C \sum_{i=1}^{n} w_{y_i} \Big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big]. \tag{5}$$

La frontière de décision est un hyperplan : le modèle est simple et interprétable, mais exposé au *biais* si la séparation réelle est non linéaire.

k plus proches voisins (k-NN). Méthode non paramétrique : un vin est classé selon le vote majoritaire de ses k plus proches voisins dans l'ensemble d'apprentissage, pour la distance de Minkowski

$$d_p(\mathbf{x}, \mathbf{x}') = \Big(\sum_{j=1}^{p} |x_j - x'_j|^p \Big)^{1/p}, \tag{6}$$

avec $p = 1$ (Manhattan) ou $p = 2$ (euclidienne). La complexité diminue quand k augmente : un petit k entraîne une forte variance. La méthode est sensible à l'échelle (d'où la standardisation) et, n'admettant pas de pondération des classes, au déséquilibre.

Arbre de décision. L'espace est partitionné récursivement par des tests de la forme $x_j \leq s$. À chaque nœud, le test retenu maximise le gain d'information (critère des algorithmes ID3/C4.5), c'est-à-dire la diminution d'entropie :

$$H(D) = -\sum_{c} p_c \log_2 p_c, \qquad \mathrm{Gain}(D, T) = H(D) - \sum_{v} \frac{|D_v|}{|D|} H(D_v). \tag{7}$$

L'arbre est interprétable et insensible à l'échelle. Non contraint, il peut cependant isoler chaque exemple dans une feuille pure : il sur-apprend.

Naive Bayes gaussien. Par le théorème de Bayes et sous l'hypothèse d'indépendance conditionnelle des variables,

$$P(y = c \mid \mathbf{x}) \ \propto \ P(y = c) \prod_{j=1}^{p} \mathcal{N}(x_j;\, \mu_{jc}, \sigma_{jc}^2). \tag{8}$$

Le modèle est très rapide et peu sujet au sur-apprentissage, mais ses deux hypothèses (gaussianité, indépendance) sont violées ici (section 5).

Machine à vecteurs de support (SVM). Le SVM à marge souple cherche l'hyperplan de marge maximale en tolérant des erreurs via des variables d'écart ξ_i :

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} w_{y_i}\, \xi_i \quad \text{s.c.} \quad \tilde{y}_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \ \ \xi_i \geq 0, \tag{9}$$

avec $\tilde{y}_i \in \{-1, +1\}$. Le noyau gaussien (RBF) $K(\mathbf{x}, \mathbf{x}') = \exp\big(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\big)$ permet une frontière non linéaire. C règle le compromis entre largeur de marge et erreurs d'apprentissage, γ la portée de l'influence de chaque exemple ; de grandes valeurs de l'un ou l'autre augmentent le risque de sur-apprentissage.

Configuration par défaut. Les modèles sont d'abord évalués avec les paramètres par défaut de scikit-learn [2], en ajoutant `class_weight="balanced"` pour la régression logistique, l'arbre et le SVM, et `criterion="entropy"` pour l'arbre.

9 Évaluation des modèles par validation croisée

9.1 Métriques

Avec la classe « bon » comme classe positive, on note TP, FP, TN, FN les effectifs de la matrice de confusion. Les métriques utilisées sont :

$$\text{Exactitude} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Précision} = \frac{TP}{TP + FP}, \tag{10}$$

$$\text{Rappel} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot \text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}. \tag{11}$$

L'AUC est l'aire sous la courbe ROC (taux de vrais positifs en fonction du taux de faux positifs, pour tous les seuils de décision). Elle s'interprète comme la probabilité qu'un bon vin tiré au hasard reçoive un score supérieur à celui d'un vin non bon tiré au hasard (0,5 : hasard ; 1 : classement parfait).

9.2 Résultats des modèles par défaut

Le tableau 3 rapporte les performances hors-pli sur l'ensemble d'apprentissage.

Table 3 – Performances des modèles par défaut (validation croisée 5 plis, prédictions hors-pli sur l'apprentissage). Le F_1 train est mesuré après réapprentissage sur tout l'ensemble d'apprentissage.

Modèle	Exactitude	Précision	Rappel	F_1	AUC	F_1 train
Régression logistique	0,742	0,407	0,793	0,538	0,832	0,537
k-NN ($k = 5$)	**0,820**	**0,533**	0,410	0,464	0,786	0,645
Arbre de décision	0,776	0,409	0,403	0,406	0,633	1,000
Naive Bayes	0,741	0,392	0,664	0,493	0,776	0,496
SVM (RBF)	0,745	0,412	**0,804**	**0,545**	**0,844**	0,577

Le SVM obtient le meilleur F_1 et la meilleure AUC, suivi de près par la régression logistique. Le k-NN illustre le piège de l'exactitude : il obtient la meilleure exactitude (0,820) mais ne retrouve que 41 % des bons vins. L'arbre non contraint est le moins bon ($F_1 = 0{,}41$, AUC $= 0{,}63$).

Matrices de confusion. La figure 7 fait apparaître deux comportements. La régression logistique et le SVM, pondérés, retrouvent environ 80 % des 807 bons vins (640 et 649), au prix de plus de 900 fausses alertes. Le k-NN et l'arbre sont plus conservateurs (respectivement 290 et 470 fausses alertes) mais manquent environ 60 % des bons vins. Le choix entre ces profils dépend de l'usage : un caviste privilégierait la précision, un sélectionneur souhaitant ne manquer aucun bon vin privilégierait le rappel.

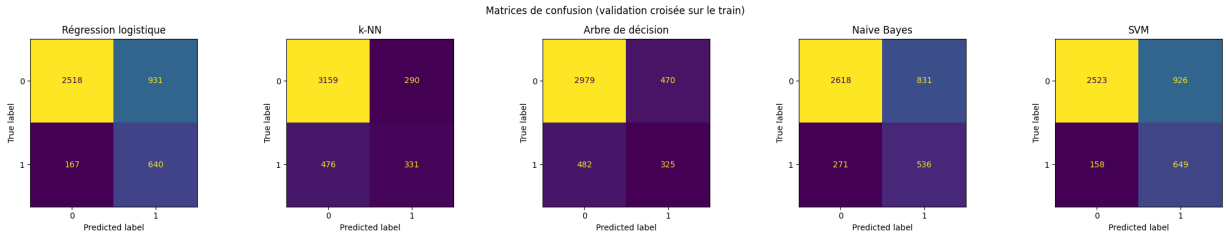


Figure 7 – Matrices de confusion hors-pli des modèles par défaut (lignes : classe réelle ; colonnes : classe prédite).

Courbes ROC. La courbe du SVM domine, celle de la régression logistique est très proche (figure 8). La courbe de l'arbre présente un unique coude : ses feuilles pures ne produisent que des probabilités 0 ou 1, ce qui est un symptôme de sur-apprentissage.

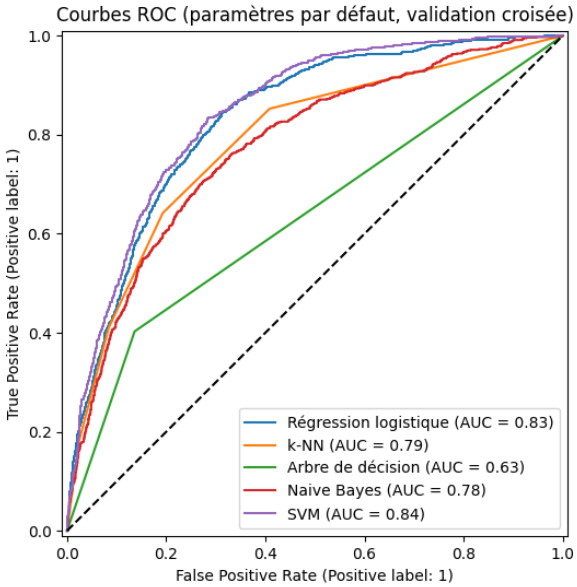


Figure 8 – Courbes ROC hors-pli des modèles par défaut.

9.3 Diagnostic biais / variance

L'erreur de généralisation se décompose en biais, variance et erreur irréductible. Le biais correspond à un modèle trop simple, qui se trompe sur l'apprentissage comme en validation (sous-apprentissage). La variance correspond à un modèle trop sensible aux données d'apprentissage, très performant sur celles-ci mais beaucoup moins sur des données nouvelles (sur-apprentissage). Nous comparons donc le F_1 sur l'apprentissage et le F_1 en validation croisée (tableau 4).

Table 4 – Diagnostic biais/variance (modèles par défaut). Le F_1 de validation est la moyenne des 5 plis.

Modèle	F_1 train	F_1 validation	Écart-type	Écart	Diagnostic
Régression logistique	0,537	0,539	0,026	−0,002	biais
k-NN	0,645	0,464	0,007	0,181	variance
Arbre de décision	1,000	0,405	0,024	0,595	forte variance
Naive Bayes	0,496	0,493	0,013	0,003	biais
SVM (RBF)	0,577	0,545	0,017	0,032	bon compromis

La régression logistique et Naive Bayes présentent des scores d'apprentissage et de validation quasi identiques mais modestes : leur limite est structurelle (frontière linéaire, hypothèses trop

fortes), ce qui relève du biais. L'arbre atteint $F_1 = 1$ en apprentissage mais seulement 0,405 en validation : il a mémorisé l'ensemble d'apprentissage. Le SVM combine le meilleur score de validation et un faible écart.

Courbes de complexité. Pour les deux modèles dont la complexité se règle par un unique paramètre, nous traçons le F_1 d'apprentissage et de validation en fonction de cette complexité (figure 9). Pour l'arbre, le F_1 d'apprentissage croît vers 1 avec la profondeur tandis que celui de validation décroît : l'optimum est atteint pour une profondeur de 3 ($F_1 = 0{,}511$). On retrouve la forme classique de l'erreur en fonction de la complexité, avec une zone de biais à gauche et une zone de variance à droite. Pour le k-NN, le F_1 de validation *diminue* quand k augmente. Sans pondération, un grand nombre de voisins fait presque toujours l'emporter la classe majoritaire, et le modèle détecte de moins en moins de bons vins. Le k-NN est ainsi pris entre la variance (petit k) et l'effet du déséquilibre (grand k).

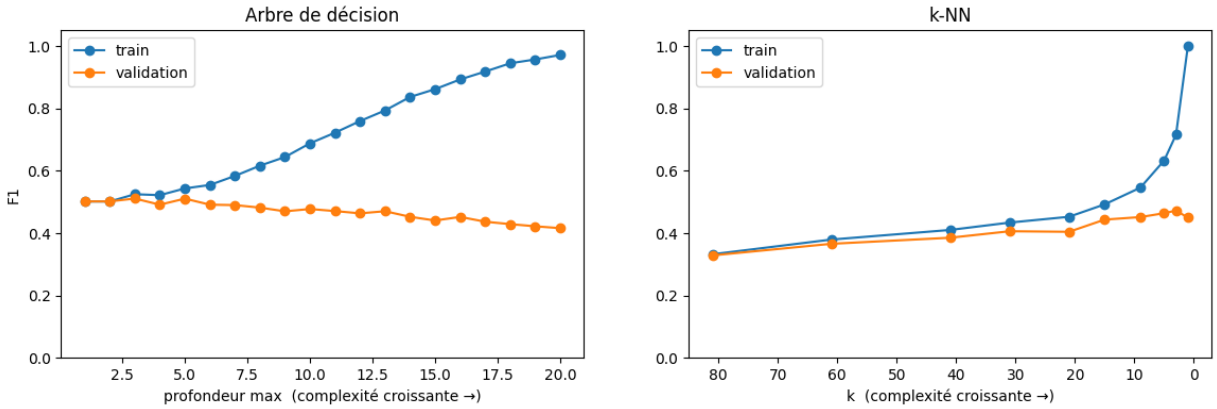


Figure 9 – F_1 d'apprentissage et de validation selon la complexité : profondeur maximale de l'arbre (gauche) et nombre de voisins k du k-NN, axe inversé (droite).

10 Optimisation des hyperparamètres

10.1 Méthodologie

Les hyperparamètres, fixés avant l'apprentissage, contrôlent pour la plupart la complexité des modèles, donc le compromis biais/variance diagnostiqué ci-dessus. Deux stratégies sont employées, chaque configuration étant évaluée par validation croisée à 5 plis sur l'apprentissage avec le F_1 comme critère :

— la **recherche en grille** (`GridSearchCV`), exhaustive, adaptée lorsque l'espace de recherche est petit ;

— la **recherche aléatoire** (`RandomizedSearchCV`), qui tire un nombre fixé de configurations et réduit le coût de calcul lorsque l'espace est grand ou le modèle coûteux.

Le choix du F_1 comme critère est essentiel : optimiser l'exactitude pousserait vers des modèles prédisant systématiquement la classe majoritaire. Le tableau 5 détaille les espaces explorés. La recherche en grille est utilisée pour la régression logistique et l'arbre, dont les grilles sont petites, et la recherche aléatoire pour le SVM, modèle le plus coûteux à entraîner. Les deux stratégies sont appliquées au k-NN afin de les comparer. Naive Bayes gaussien n'est pas optimisé : il ne possède pas d'hyperparamètre structurant, ses seuls paramètres (moyennes et variances par classe) étant estimés à partir des données.

Table 5 – Espaces de recherche des hyperparamètres.

Modèle	Méthode	Espace de recherche	Configurations
Régression logistique	grille	$C \in \{10^{-3}, \ldots, 10^{2}\}$, `class_weight` $\in$ {aucun, balanced}	12
Arbre (entropie)	grille	profondeur $\in \{3, 5, 7, 10, 15, \infty\}$, taille min. des feuilles $\in \{1, 5, 10, 20, 50\}$, `class_weight`	60
k-NN	grille	$k \in \{1, 3, 5, 7, 9, 11, 15, 21, 31\}$, $p \in \{1, 2\}$	18
	aléatoire	$k \in \{1, \ldots, 39\}$, $p \in \{1, 2\}$	10 tirées sur 78
SVM (RBF)	aléatoire	$C \in \{0{,}1;\ 0{,}3;\ 1;\ 3;\ 10;\ 30;\ 100\}$, $\gamma \in \{0{,}001;\ 0{,}003;\ \ldots;\ 1\}$	20 tirées sur 49

10.2 Résultats

Table 6 – Hyperparamètres retenus et F_1 moyen en validation croisée avant et après optimisation.

Modèle	Configuration retenue	F_1 défaut	F_1 optimisé
Régression logistique	$C = 0{,}01$, balanced	0,539	0,540
Arbre de décision	profondeur 3, feuille min. 1, balanced	0,405	**0,511**
k-NN	$k = 1$, Manhattan ($p = 1$)	0,464	0,482
Naive Bayes	–	0,493	0,493
SVM (RBF)	$C = 100$, $\gamma = 0{,}01$, balanced	**0,545**	**0,544**

L'**arbre** est le modèle qui bénéficie le plus de l'optimisation (+0,11). La profondeur retenue (3) coïncide avec l'optimum de la courbe de complexité : limiter la profondeur supprime l'essentiel de la variance. La **régression logistique** ne progresse pratiquement pas malgré une régularisation forte ($C = 0{,}01$), ce qui confirme que sa limite est un biais structurel. Pour le **SVM**, la configuration trouvée égale la configuration par défaut ($\gamma = \texttt{scale} = 1/p \approx 0{,}083$ sur données standardisées, $C = 1$) : celle-ci se situait déjà dans une zone favorable. Pour le **k-NN**, les deux recherches aboutissent à la même configuration ($k = 1$, distance de Manhattan), la recherche aléatoire avec presque deux fois moins d'évaluations. Ce résultat appelle toutefois une réserve : $k = 1$ est le cas de variance maximale. Le critère F_1 favorise les petites valeurs de k parce qu'en l'absence de pondération, c'est la seule manière pour le k-NN de prédire suffisamment de bons vins. Enfin, la pondération `balanced` est retenue pour la régression logistique comme pour l'arbre, ce qui confirme la nécessité de traiter le déséquilibre.

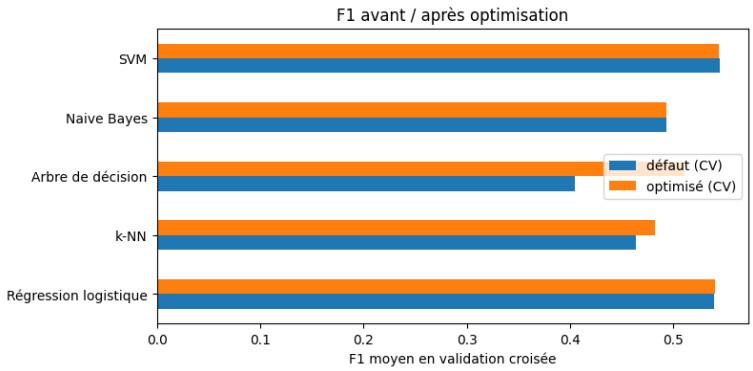


Figure 10 – F_1 moyen en validation croisée avant et après optimisation.

11 Évaluation finale sur le jeu de test

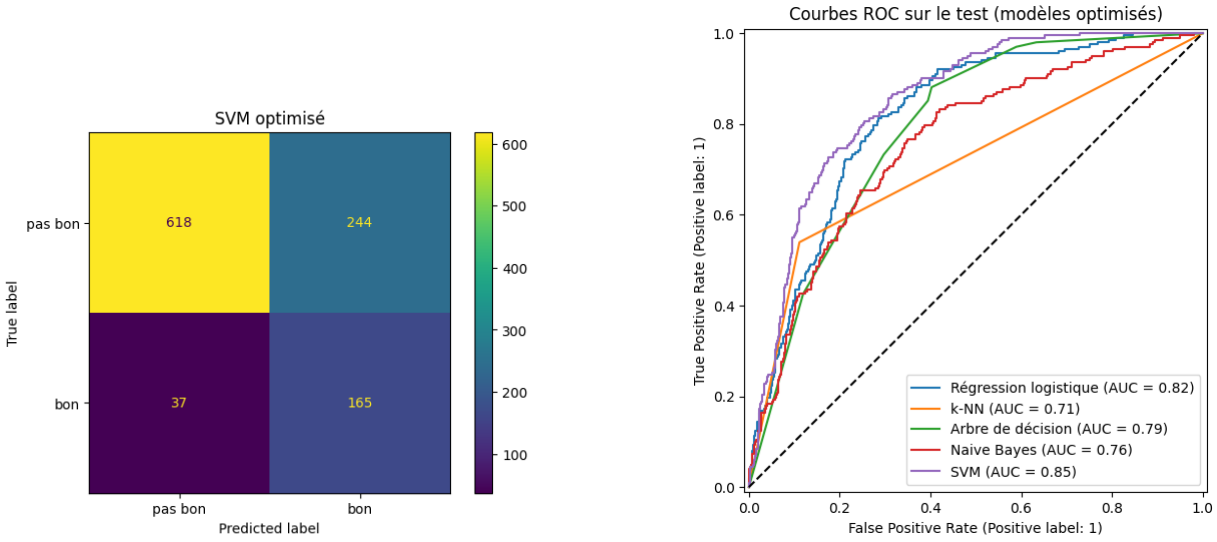
Le modèle final est choisi sur la base du F_1 en validation croisée, *avant* toute utilisation du test. Les cinq modèles optimisés sont ensuite réentraînés sur l'ensemble d'apprentissage complet et évalués une seule fois sur le test (tableau 7).

Table 7 – Évaluation finale des modèles optimisés sur le jeu de test (1 064 vins, dont 202 bons), triés par F_1 de validation croisée.

Modèle	F_1 CV	Exactitude	Précision	Rappel	F_1 test	AUC	F_1 train
SVM (RBF)	**0,544**	0,736	0,403	**0,817**	**0,540**	**0,850**	0,573
Régression logistique	0,540	0,735	0,400	0,797	0,533	0,817	0,541
Arbre de décision	0,511	0,709	0,366	0,733	0,488	0,791	0,525
Naive Bayes	0,493	0,733	0,382	0,653	0,482	0,759	0,496
k-NN	0,482	**0,822**	**0,532**	0,540	0,536	0,714	1,000

Pour le SVM, la régression logistique et Naive Bayes, les scores de test sont très proches des scores de validation croisée (écarts inférieurs à 0,012), ce qui atteste de la fiabilité du protocole. Le k-NN obtient un F_1 de test (0,536) nettement supérieur à son F_1 de validation (0,482). Associé à la plus faible AUC (0,714) et à un F_1 d'apprentissage de 1, ce résultat relève vraisemblablement de la variabilité d'échantillonnage. Il ne peut de toute façon pas servir à choisir le modèle sans réutiliser le test.

Le **SVM à noyau RBF** est retenu comme modèle final : il est le meilleur en validation croisée comme sur le test ($F_1 = 0{,}540$, AUC $= 0{,}850$) et présente le meilleur compromis biais/variance. Sur le test, il retrouve 165 des 202 bons vins (rappel de 82 %), au prix de 244 fausses alertes sur 862 vins non bons (précision de 40 %), comme le montre la figure 11.



(a) Matrice de confusion du SVM.

(b) Courbes ROC des modèles optimisés.

Figure 11 – Évaluation finale sur le jeu de test.

Interprétation géométrique. Pour visualiser le comportement du SVM, un modèle de mêmes hyperparamètres est entraîné sur les deux variables les mieux classées par la RFE, l'alcool et l'acidité volatile (figure 12). Il s'agit d'une illustration et non du modèle final, qui utilise les douze variables. Quatre observations ressortent :

- les bons vins se concentrent dans la région « alcool élevé, acidité volatile faible à modérée » ;
- la frontière est courbe et oblique : plus l'acidité volatile est élevée, plus la teneur en alcool requise pour être classé « bon » augmente, ce qu'un modèle linéaire ne peut pas représenter ;
- les deux classes se chevauchent largement, ce qui borne les performances atteignables ;
- la région « bon » est étendue du fait de la pondération des classes, ce qui explique le rappel élevé et la précision modeste.

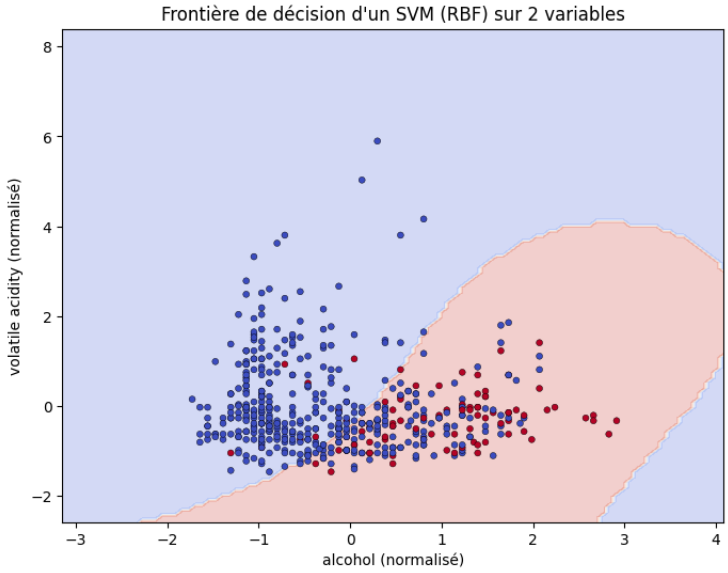


Figure 12 – Frontière de décision d'un SVM RBF ($C = 100$, $\gamma = 0{,}01$) entraîné sur l'alcool et l'acidité volatile (variables standardisées). Zone rouge : prédiction « bon ». Points : 500 vins de l'apprentissage.

12 Discussion

Niveau de performance. Un F_1 de 0,54 peut paraître modeste. Plusieurs facteurs l'expliquent. La note est une médiane d'appréciations subjectives, et la frontière à 7 sépare des vins (notés 6 et 7) souvent très proches chimiquement. Les variables disponibles ne décrivent pas tous les déterminants de la qualité : cépage, millésime, arômes ou prix n'y figurent pas. Enfin, la figure 12 montre un recouvrement important des classes. L'AUC de 0,85 indique néanmoins que le modèle ordonne correctement les vins dans la grande majorité des cas.

Compromis précision/rappel. Le modèle final privilégie le rappel. Ce compromis découle de la pondération des classes et du seuil de décision par défaut. Selon l'usage (recommandation à un client contre présélection pour une dégustation), un autre point de fonctionnement sur la courbe ROC pourrait être préférable. Ajuster le seuil par validation croisée constitue d'ailleurs une alternative à la pondération des classes, applicable aux cinq modèles, y compris le k-NN et Naive Bayes qui n'admettent pas `class_weight`.

Limites méthodologiques.

— La standardisation est ajustée sur l'ensemble d'apprentissage complet avant la validation croisée. Chaque pli de validation contribue donc marginalement au calcul des moyennes et écarts-types. L'effet est négligeable ici, mais l'usage d'un `Pipeline` scikit-learn l'éliminerait.

— La RFE repose sur un modèle linéaire. Une sélection fondée sur un modèle non linéaire pourrait produire un classement différent.

— Les performances proviennent d'un unique découpage apprentissage/test. Une validation croisée répétée ou imbriquée fournirait des intervalles de confiance plus fiables, en particulier pour départager le SVM et la régression logistique, dont les scores diffèrent de moins d'un écart-type.

— La binarisation au seuil 7 fait perdre l'information ordinale de la note.

Perspectives. Plusieurs pistes peuvent être explorées :

— les méthodes d'ensemble : forêts aléatoires (*bagging*), AdaBoost (*boosting*) ou empilement (*stacking*) des modèles étudiés, généralement plus performantes que les modèles individuels sur ce type de données tabulaires ;

— l'ajustement du seuil de décision à partir de la courbe ROC ;

— des modèles distincts pour les vins rouges et blancs ;

— une formulation en régression ou en classification ordinale exploitant la note complète.

13 Conclusion

Ce projet a mis en œuvre une chaîne complète d'apprentissage supervisé pour prédire la qualité d'un vin à partir de mesures physico-chimiques. Le déséquilibre des classes (19 % de bons vins) a structuré les choix méthodologiques : découpage stratifié, pondération des classes, score F_1 et AUC comme critères. L'analyse exploratoire et la sélection par RFE convergent sur le rôle prépondérant du degré d'alcool et de l'acidité volatile. La comparaison des cinq modèles, menée exclusivement par validation croisée, a permis d'identifier les sources d'erreur de chacun : biais pour la régression logistique et Naive Bayes, variance pour l'arbre non contraint et le k-NN. L'optimisation des hyperparamètres a surtout bénéficié à l'arbre, dont la profondeur limitée corrige le sur-apprentissage. Évalué une seule fois sur le jeu de test, le SVM à noyau RBF obtient les meilleures performances ($F_1 = 0{,}54$, AUC $= 0{,}85$, rappel $= 0{,}82$), avec un écart minime entre validation et test. Ses performances restent néanmoins limitées par la subjectivité de la note et le recouvrement des classes. Les méthodes d'ensemble et l'ajustement du seuil de décision constituent les prolongements les plus prometteurs.

Références

[1] P. Cortez, A. Cerdeira, F. Almeida, T. Matos et J. Reis. Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4) :547–553, 2009.

[2] F. Pedregosa *et al.* Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12 :2825–2830, 2011.

[3] J. W. Tukey. *Exploratory Data Analysis*. Addison-Wesley, 1977.

[4] A. G. Blaiech, A. Tay et H. Shraim. *Cours de Machine Learning I*, chapitres I à VI. EFREI Paris, 2026–2027.

A Reproductibilité

L'ensemble des traitements est contenu dans le notebook `projet_vin.ipynb` (Python, pandas, scikit-learn, matplotlib, seaborn). Les sources d'aléa sont fixées (`random_state=42`) pour le découpage, l'arbre, le SVM et les recherches aléatoires. Les validations croisées utilisent un découpage stratifié à 5 plis (`cv=5`). Les fichiers `winequality-red.csv` et `winequality-white.csv` doivent être placés dans le dossier `data/` du dépôt.